

LABORATORIO 04

Buscar sin dar vuelta el almacén

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo. Las salidas son las de la solución ejecutada del repositorio.

Introducción

1 · El tema

Una analista necesita el total de un solo mes, y la tabla de documentos tiene un año entero. En este laboratorio vas a medir cuánto tiene que leer el motor para contestarle, y vas a ver tres ideas que cambian esa cuenta. Particionar, el particionamiento oculto y la evolución de particiones.

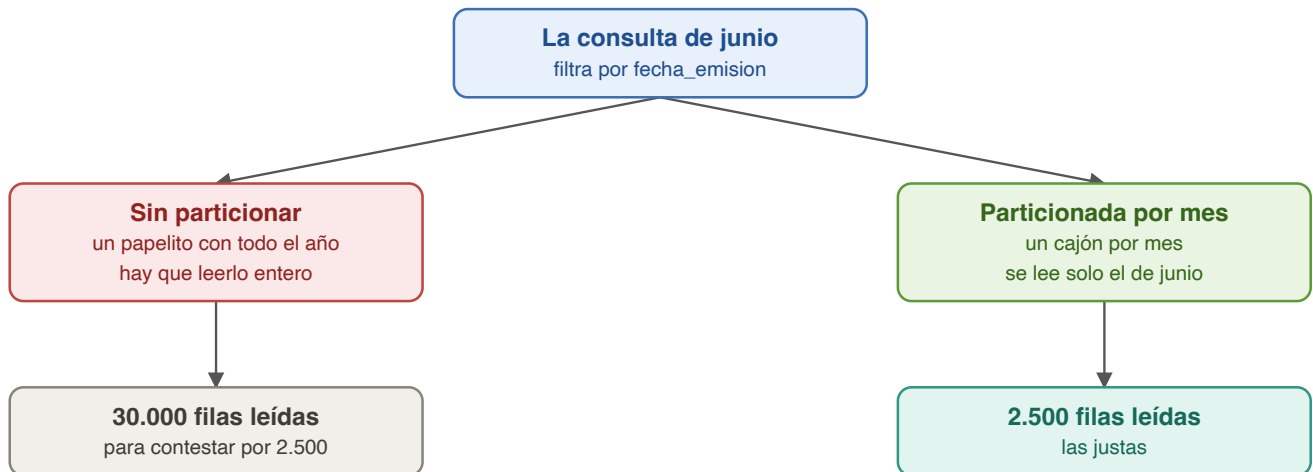
2 · El problema, en el almacén

En una bodega sin orden, para encontrar los fideos hay que abrir todas las cajas. Si alguien pregunta por lo que llegó en junio, el bodeguero da vuelta el almacén entero, aunque lo que busca esté en un rincón.

3 · El problema en términos técnicos

Una tabla es un conjunto de archivos. Cuando una consulta pide un mes, el motor solo puede saltarse un archivo si sabe, sin abrirlo, que ese archivo no tiene nada de ese mes. Si todos los documentos del año quedaron mezclados en un mismo archivo, no hay nada que saltar, y responder por un mes cuesta lo mismo que responder por el año completo. Con dos millones de documentos y años de historia, esa es la diferencia entre un reporte que sale y uno que no.

4 · El diagrama



La consulta de junio (azul). La analista pide el total de junio y filtra por la fecha de emisión, la columna de verdad. No sabe nada de cómo está guardada la tabla.

Sin particionar (rojo). La tabla tiene un solo papelito con el año completo. Junio puede estar en cualquier parte de él, así que hay que leerlo entero.

Particionada por mes (verde). La tabla guarda los documentos en un cajón por mes. El motor va derecho al cajón de junio y no abre los otros once.

30.000 filas leídas (gris). Lo que cuesta sin particionar, el año entero para contestar por un mes.

2.500 filas leídas (verde agua). Lo que cuesta particionada, solo las filas de junio.

5 · La solución

En el almacén, la bodega se ordena en pasillos rotulados, uno por mes. Quien pide fideos de junio no necesita saber cómo está ordenada la bodega. Pide lo que quiere y el bodeguero sabe a qué pasillo ir. Y si un día un pasillo queda chico y hay que dividirlo por día, lo que ya estaba guardado se queda donde está y solo lo nuevo se guarda con el orden nuevo.

En Iceberg eso son tres cosas. **Particionar** con `PARTITIONED BY (months(fecha_emision))`, que guarda los documentos en un cajón por mes. El **particionamiento oculto**, que permite filtrar por la fecha real sin nombrar el mes, porque Iceberg sabe cómo se deriva el cajón a partir de la fecha. Y la **evolución de particiones**, que cambia el criterio de aquí en adelante con una sentencia, sin reescribir lo que ya estaba.

6 · Los pasos

- **Paso 0.** Creas una tabla con un año de documentos, sin particionar, y la miras.
- **Paso 1.** Haces la consulta de junio y mides cuántos archivos y filas tiene que leer.
- **Paso 2.** Creas la misma tabla particionada por mes.
- **Paso 3.** Repites la consulta y la medición sobre la tabla nueva.
- **Paso 4.** Miras cómo quedó particionada y los cajones que se armaron.
- **Paso 5.** Cambias el criterio de mes a día, cargas un mes nuevo y compruebas que los dos criterios conviven.
- **Cierre.** Lo que leíste en cada caso, lado a lado.

Las celdas 0.2 y 0.3 borran las dos tablas del laboratorio si existen, así que se puede repetir desde arriba. Si prefieres dejar todo limpio antes de empezar, corre el comando de abajo desde la carpeta del repositorio.

```
bash
bin/reiniciar-lab.sh 04
```

Paso 0 · Preparar la tabla

Celda 0.1 · Ponerte en tu espacio

La pregunta. ¿En qué espacio de trabajo van a quedar las tablas que crees?

Por qué ahora. Las dos tablas del laboratorio tienen que quedar en tu espacio, `mi_espacio`. Esta celda te pone ahí.

En el almacén. Es pararte frente a tu propio mostrador antes de abrir la libreta.

La sentencia, parte por parte.

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 0.1` es un comentario. Todo lo que va después de dos guiones lo ignora el motor.
- `USE mi_espacio` deja a `mi_espacio` como espacio de trabajo. Una tabla nombrada sin espacio adelante se busca y se crea ahí.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:06:43 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. Ya estás en tu espacio. `USE` no devuelve filas, por eso el cuaderno solo avisa que se ejecutó.

Las tres primeras líneas no son errores. Salen solo en la primera celda que usa Spark, cuando arranca.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores, no todo lo que hace.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. No hace falta tocarlo. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y que usa la versión en Java. Funciona igual. `26/09/25 18:06:43` es la fecha y la hora del aviso, año 2026, mes 09, día 25, en hora UTC.

Celda 0.2 · Borrar la primera tabla

La pregunta. ¿Qué pasa si la tabla sin particionar ya existe de una vez anterior?

Por qué ahora. Hoy se crean dos tablas. Si el laboratorio se corrió antes, la creación fallaría porque el nombre ya existe. Esta celda borra la primera.

En el almacén. Es sacar del mostrador la libreta vieja de este ejercicio, si quedó alguna.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla.
- `IF EXISTS` hace que no falle si la tabla no existe.
- `documentos_sin_particion` es la tabla sin particionar, en `mi_espacio`.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS documentos_sin_particion
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó sin problema, existiera o no la tabla.

Celda 0.3 · Borrar la segunda tabla

La pregunta. ¿Y la tabla particionada, que se crea más adelante?

Por qué ahora. Por la misma razón que la celda anterior, se borra antes de empezar.

En el almacén. Es sacar también la segunda libreta vieja.

La sentencia, parte por parte.

- `DROP TABLE IF EXISTS` borra la tabla si existe y no hace nada si no.
- `documentos_por_mes` es la tabla particionada del paso 2.

```
sql
%%sql
-- Celda 0.3
DROP TABLE IF EXISTS documentos_por_mes
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó sin problema.

Celda 0.4 · Crear la tabla sin particionar

La pregunta. ¿Cómo queda un año de documentos si nadie piensa en cómo guardarlo?

Por qué ahora. Es el punto de comparación. Todo lo que se mida después se compara con esta tabla.

En el almacén. Es echar todas las facturas del año en una sola caja grande, sin orden.

La sentencia, parte por parte.

- `CREATE TABLE documentos_sin_particion` crea la tabla con ese nombre.
- `USING iceberg` hace que sea una tabla Iceberg.
- `AS SELECT * FROM curso.dte_2024` le da las columnas de esa consulta y la carga con sus filas. `curso.dte_2024` es un año completo de documentos, 2024, en el espacio `curso` de las tablas de referencia. Tú solo lo lees.

```
sql
%%sql
-- Celda 0.4
CREATE TABLE documentos_sin_particion USING iceberg AS
SELECT * FROM curso.dte_2024
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla quedó creada y cargada.

Celda 0.5 · Mirar cinco documentos

La pregunta. ¿Qué columnas trae un documento?

Por qué ahora. Antes de medir conviene saber con qué columnas se trabaja, sobre todo cuál es la fecha que se va a filtrar.

En el almacén. Es leer las primeras facturas de la caja.

La sentencia, parte por parte.

- `SELECT *` pide todas las columnas.
- `FROM documentos_sin_particion` es la tabla recién creada.
- `ORDER BY fecha_emision, rut_emisor, tipo_dte, folio` ordena por fecha de emisión y, dentro de la misma fecha, por emisor, tipo y folio, para que el resultado salga siempre igual.
- `LIMIT 5` se queda con las cinco primeras.

```
sql
%%sql
-- Celda 0.5
SELECT * FROM documentos_sin_particion
ORDER BY fecha_emision, rut_emisor, tipo_dte, folio
LIMIT 5
```

Lo que sale en pantalla.

```
text
rut_emisor  razon_social_emisor  tipo_dte  folio  fecha_emision  fecha_recepcion  monto_netto  monto_iva  monto_total  estado_sii
76114218-6  Coihue Maquinarias SpA    33        1      2024-01-01    2024-01-01 13:46:49  1656950.00  314820.50    1971770.50  ACEPTADO
76169126-0  Ulmo Maquinarias Ltda.    33        1      2024-01-01    2024-01-03 14:40:08  235404.00   44726.76    280130.76    ACEPTADO
76194732-K  Bolfo Importadora Ltda.   39        1      2024-01-01    2024-01-02 01:42:09  113373.00   21540.87    134913.87    ACEPTADO
76253726-5  Ulmo Transportes S.A.     33        4      2024-01-01    2024-01-03 13:28:12  8458882.00  1607187.58  10066069.58  ACEPTADO
76257050-5  Araucaria Agrícola Ltda.  39       11      2024-01-01    2024-01-02 09:49:18  41107.00    7810.33     48917.33     ACEPTADO
5 filas.
```

Cómo se lee. Cinco documentos del 1 de enero de 2024, uno por fila.

- `rut_emisor` y `razon_social_emisor` son el RUT y el nombre de quien emitió el documento.
- `tipo_dte` es el tipo de documento. 33 es factura afecta y 39 boleta.
- `folio` es el número del documento.
- `fecha_emision` es el día en que se emitió. Es la columna que se filtra en todo el laboratorio.
- `fecha_recepcion` es cuándo lo recibió la DGT, con hora.
- `monto_netto`, `monto_iva` y `monto_total` son los montos. El IVA es el 19 por ciento del neto, y el total es la suma de los dos.
- `estado_sii` es el estado con que quedó el documento.

Celda 0.6 · Contar los documentos

La pregunta. ¿Cuántos documentos quedaron en la tabla?

Por qué ahora. Es el total contra el que se comparan todas las lecturas del laboratorio.

En el almacén. Es contar las facturas de la caja.

La sentencia, parte por parte.

- `SELECT count(*)` cuenta todas las filas.
- `AS documentos` le pone nombre a la columna.
- `FROM documentos_sin_particion` es la tabla.

```
sql
%%sql
-- Celda 0.6
SELECT count(*) AS documentos FROM documentos_sin_particion
```

Lo que sale en pantalla.

```
text
documentos
30000
1 fila.
```

Cómo se lee. 30.000 documentos, los doce meses de 2024.

Paso 1 · La consulta lenta

Celda 1.1 · El total de junio

La pregunta. ¿Cuántos documentos hay en junio y cuánto suman?

Por qué ahora. Es la consulta de todos los meses, la que el laboratorio quiere hacer barata.

En el almacén. Es pedir el total de las facturas de junio.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos` cuenta los documentos de junio.
- `sum(monto_total) AS total_junio` suma sus montos totales.
- `FROM documentos_sin_particion` es la tabla sin particionar.
- `WHERE fecha_emision BETWEEN DATE '2024-06-01' AND DATE '2024-06-30'` se queda con los documentos emitidos entre el 1 y el 30 de junio, los dos días incluidos. `DATE '...'` convierte el texto en una fecha.

```
sql
%%sql
-- Celda 1.1
SELECT count(*)          AS documentos,
       sum(monto_total) AS total_junio
FROM documentos_sin_particion
WHERE fecha_emision BETWEEN DATE '2024-06-01' AND DATE '2024-06-30'
```

Lo que sale en pantalla.

```
text
documentos  total_junio
2500        24547341762.59
1 fila.
```

Cómo se lee. 2500 documentos en junio, que suman 24547341762.59 pesos. La respuesta es correcta y sale rápido porque la tabla es chica. La pregunta del laboratorio no es cuánto demoró, sino cuánto tuvo que leer.

Celda 1.2 · Los papelitos de la tabla

La pregunta. ¿En cuántos archivos está guardada la tabla, y qué fechas abarca cada uno?

Por qué ahora. El motor decide qué leer mirando archivos enteros. Hay que ver cuántos hay y qué dice el rótulo de cada uno.

En el almacén. Cada papelito lleva pegado un rótulo con la fecha más antigua y la más nueva que contiene. Esta celda lee esos rótulos sin abrir los papelitos.

La sentencia, parte por parte.

- `SELECT record_count AS filas` es cuántas filas tiene cada archivo.
- `readable_metrics.fecha_emision.lower_bound AS desde` es la fecha más chica que hay en ese archivo. `readable_metrics` guarda, para cada columna, el mínimo y el máximo del archivo, que es el rótulo.
- `readable_metrics.fecha_emision.upper_bound AS hasta` es la fecha más grande.
- `FROM mi_espacio.documentos_sin_particion.files` lee la vista de sistema `.files`, con una fila por archivo de datos. El nombre va con tres partes, espacio, tabla y vista, aunque ya hayas hecho `USE`.
- `ORDER BY desde` ordena por la fecha más chica.

```
sql
%%sql
-- Celda 1.2
SELECT record_count                AS filas,
       readable_metrics.fecha_emision.lower_bound AS desde,
       readable_metrics.fecha_emision.upper_bound AS hasta
FROM mi_espacio.documentos_sin_particion.files
ORDER BY desde
```

Lo que sale en pantalla.

```
text
filas desde      hasta
30000 2024-01-01 2024-12-31
1 fila.
```

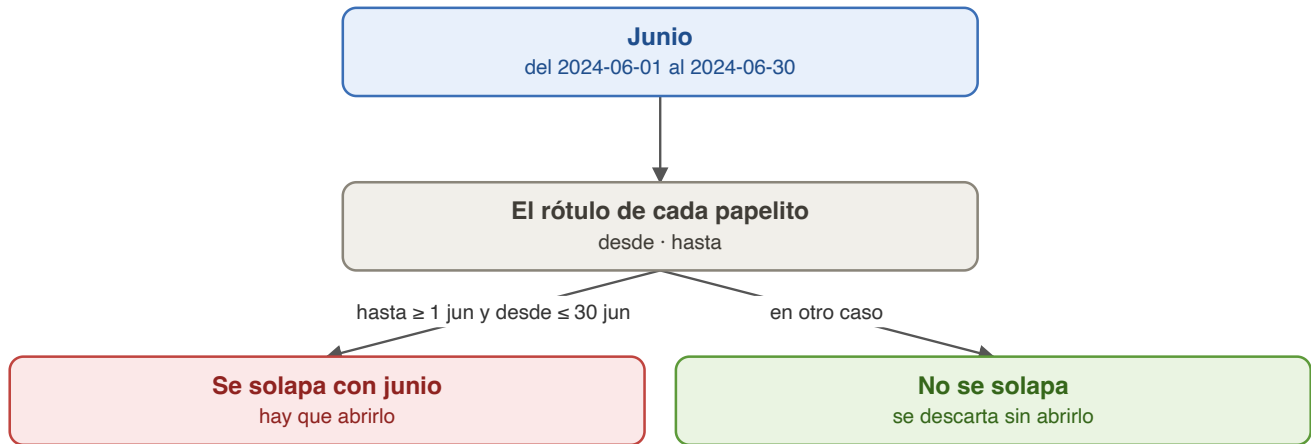
Cómo se lee. Un solo archivo, con 30000 filas, que va desde el `2024-01-01` hasta el `2024-12-31`. Junio puede estar en cualquier parte de ese archivo, así que el rótulo no permite descartarlo y hay que leerlo entero.

Celda 1.3 · Lo que no se puede descartar

La pregunta. Para contestar por junio, ¿cuántos archivos, filas y bytes tiene que leer el motor?

Por qué ahora. Esta es la medición del laboratorio. Se repite igual en el paso 3 sobre la tabla particionada.

En el almacén. Es contar los papelitos cuyo rótulo no descarta junio, los que el bodeguero igual tiene que abrir.



Junio (azul). El rango de fechas que pide la consulta.

El rótulo de cada papelito (gris). La fecha más chica y la más grande de cada archivo, que se leen de `.files` sin abrir el archivo.

Se solapa con junio (rojo). Si el archivo termina después de que empieza junio y empieza antes de que junio termine, puede tener documentos de junio y hay que abrirlo.

No se solapa (verde). Si no, el archivo no puede tener nada de junio y se descarta sin leerlo.

La sentencia, parte por parte.

- `SELECT count(*) AS archivos_a_leer` cuenta los archivos que no se pueden descartar.
- `sum(record_count) AS filas_a_leer` suma sus filas.
- `sum(file_size_in_bytes) AS bytes_a_leer` suma su tamaño en bytes.
- `FROM mi_espacio.documentos_sin_particion.files` es la vista de archivos, con sus tres partes.
- `WHERE readable_metrics.fecha_emision.upper_bound >= DATE '2024-06-01'` pide que el archivo termine en junio o después.
- `AND readable_metrics.fecha_emision.lower_bound <= DATE '2024-06-30'` pide además que empiece en junio o antes. Las dos condiciones juntas dicen que el archivo se solapa con junio, que es la misma regla que usa el motor para decidir qué abrir.

```
sql
%%sql
-- Celda 1.3
SELECT count(*)           AS archivos_a_leer,
       sum(record_count)   AS filas_a_leer,
       sum(file_size_in_bytes) AS bytes_a_leer
FROM mi_espacio.documentos_sin_particion.files
WHERE readable_metrics.fecha_emision.upper_bound >= DATE '2024-06-01'
      AND readable_metrics.fecha_emision.lower_bound <= DATE '2024-06-30'
```

Lo que sale en pantalla.

```
text
archivos_a_leer  filas_a_leer  bytes_a_leer
1                30000        776514
1 fila.
```

Cómo se lee. Hay que leer 1 archivo con 30000 filas, que pesa 0,78 MB (776514 bytes). Es el año entero para contestar por 2500 documentos.

Paso 2 · Crear la tabla particionada

Celda 2.1 · La misma tabla, con cajones por mes

La pregunta. ¿Cómo se guarda la misma tabla con un cajón por mes?

Por qué ahora. Para comparar hace falta la misma tabla, con los mismos documentos, guardada de otra forma.

En el almacén. Es repartir las facturas del año en doce cajones rotulados, uno por mes.

La sentencia, parte por parte.

- `CREATE TABLE documentos_por_mes USING iceberg` crea una tabla Iceberg nueva.
- `PARTITIONED BY (months(fecha_emision))` le dice que guarde los documentos agrupados por el mes de su fecha de emisión. `months(...)` es una transformación. No crea ninguna columna nueva. Iceberg toma la fecha real y deriva de ella el mes.
- `AS SELECT * FROM curso.dte_2024` la carga con el mismo año de documentos.
- `ORDER BY fecha_emision` hace que las filas lleguen agrupadas por fecha. Para escribir en una tabla particionada las filas tienen que llegar juntas por cajón, y sin este orden la sentencia falla con un error que habla de *records are clustered*.

```
sql
%%sql
-- Celda 2.1
CREATE TABLE documentos_por_mes USING iceberg
PARTITIONED BY (months(fecha_emision)) AS
SELECT * FROM curso.dte_2024 ORDER BY fecha_emision
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla particionada quedó creada y cargada.

Celda 2.2 · Contar la tabla nueva

La pregunta. ¿Quedaron los mismos documentos?

Por qué ahora. La comparación solo vale si las dos tablas tienen lo mismo.

En el almacén. Es contar las facturas de los doce cajones juntos.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos` cuenta todas las filas.
- `FROM documentos_por_mes` es la tabla particionada.

```
sql
%%sql
-- Celda 2.2
SELECT count(*) AS documentos FROM documentos_por_mes
```

Lo que sale en pantalla.

```
text
documentos
30000
1 fila.
```

Cómo se lee. 30000 documentos, los mismos de la tabla sin particionar.

Paso 3 · La misma consulta, ahora sí

Celda 3.1 · El total de junio otra vez

La pregunta. ¿La tabla particionada da el mismo resultado?

Por qué ahora. Antes de medir la lectura, se comprueba que la respuesta es la misma. La consulta es letra por letra igual a la del paso 1, cambiando solo el nombre de la tabla.

En el almacén. Es pedir otra vez el total de junio, ahora en la bodega ordenada.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos` cuenta los documentos de junio.
- `sum(monto_total) AS total_junio` suma sus montos.
- `FROM documentos_por_mes` es la tabla particionada.
- `WHERE fecha_emision BETWEEN DATE '2024-06-01' AND DATE '2024-06-30'` filtra por la fecha real, igual que antes. No se nombra el mes en ninguna parte.

```
sql
%%sql
-- Celda 3.1
SELECT count(*)          AS documentos,
       sum(monto_total)  AS total_junio
FROM documentos_por_mes
WHERE fecha_emision BETWEEN DATE '2024-06-01' AND DATE '2024-06-30'
```

Lo que sale en pantalla.

```
text
documentos  total_junio
2500        24547341762.59
1 fila.
```

Cómo se lee. 2500 documentos y el mismo total, `24547341762.59`. Los datos son los mismos, así que la respuesta también.

Celda 3.2 · Medir la lectura

La pregunta. Para contestar por junio, ¿cuánto tiene que leer ahora el motor?

Por qué ahora. Es la misma medición de la celda 1.3, sobre la tabla particionada. Es a lo que vino el laboratorio.

En el almacén. Es contar los cajones que el bodeguero tiene que abrir para junio.

La sentencia, parte por parte.

- `SELECT count(*) AS archivos_a_leer, sum(record_count) AS filas_a_leer, sum(file_size_in_bytes) AS bytes_a_leer` cuenta archivos, filas y bytes que no se pueden descartar.
- `FROM mi_espacio.documentos_por_mes.files` es la vista de archivos de la tabla particionada.
- `WHERE readable_metrics.fecha_emision.upper_bound >= DATE '2024-06-01' AND readable_metrics.fecha_emision.lower_bound <= DATE '2024-06-30'` es la misma regla de solapamiento con junio.

```

sql
%%sql
-- Celda 3.2
SELECT count(*)           AS archivos_a_leer,
       sum(record_count)   AS filas_a_leer,
       sum(file_size_in_bytes) AS bytes_a_leer
FROM mi_espacio.documentos_por_mes.files
WHERE readable_metrics.fecha_emision.upper_bound >= DATE '2024-06-01'
      AND readable_metrics.fecha_emision.lower_bound <= DATE '2024-06-30'

```

Lo que sale en pantalla.

```

text
archivos_a_leer  filas_a_leer  bytes_a_leer
1               2500        71779
1 fila.

```

Cómo se lee. Hay que leer 1 archivo con 2500 filas, que pesa 0,07 MB (71779 bytes). Contra 30000 filas de la tabla sin particionar, son doce veces menos.

La tabla particionada tiene más archivos en total, doce, uno por mes. Pero para esta consulta abre uno solo. Más archivos y menos lectura.

Paso 4 · El particionamiento oculto

Celda 4.1 · Cómo está particionada

La pregunta. ¿Qué dice la tabla sobre cómo está particionada?

Por qué ahora. La consulta del paso 3 nunca nombró el mes y aun así el motor fue a un solo cajón. Hay que ver dónde está guardada esa regla.

En el almacén. Es leer en la portada de la libreta cómo están rotulados los cajones.

La sentencia, parte por parte.

- `DESCRIBE TABLE EXTENDED` pide la descripción completa de la tabla, con sus columnas, su partición y sus propiedades.
- `documentos_por_mes` es la tabla particionada.

```
sql
%%sql
-- Celda 4.1
DESCRIBE TABLE EXTENDED documentos_por_mes
```

Lo que sale en pantalla.

```
text
col_name      data_type      comment
rut_emisor    string
razon_social_emisor string
tipo_dte      int
folio         bigint
fecha_emision date
fecha_recepcion timestamp
monto_neto    decimal(18,2)
monto_iva     decimal(18,2)
monto_total   decimal(18,2)
estado_sii    string

# Partitioning
Part 0         months(fecha_emision)

# Metadata Columns
_spec_id      int
_partition    struct<fecha_emision_month:int>
_file         string
_pos          bigint
_deleted      boolean

# Detailed Table Information
Name          spark_catalog.mi_espacio.documentos_por_mes
Location      hdfs://namenode:8020/warehouse/iceberg/mi_espacio.db/documentos_por_mes
Provider      iceberg
Owner         root
Table Properties [current-snapshot-id=8267141702760514426,format=iceberg/parquet,format-version=1]
27 filas.
```

Cómo se lee. Lo importante es la línea `Part 0`, que dice `months(fecha_emision)`. Esa es toda la regla de partición. Y en las columnas de arriba no hay ninguna columna `mes`. El mes no se guarda como columna, se deriva de la fecha, y por eso se llama particionamiento oculto.

Ahora cada bloque.

- Las diez primeras filas son las columnas y sus tipos. `string` es texto, `int` y `bigint` son enteros, `date` una fecha, `timestamp` fecha con hora y `decimal(18,2)` un número de hasta 18 dígitos con 2 decimales.
- `# Partitioning` es la regla de partición, con `Part 0` como único criterio.
- `# Metadata Columns` son columnas internas que Iceberg agrega a cada fila y que se pueden consultar. `_spec_id` es el número del criterio de partición con que se escribió la fila, `_partition` el cajón de la fila, `_file` el archivo donde está, `_pos` su posición dentro del archivo y `_deleted` si está borrada. `struct<fecha_emision_month:int>` quiere decir que el cajón se guarda como un número entero llamado `fecha_emision_month`.
- `# Detailed Table Information` trae el nombre completo, `spark_catalog.mi_espacio.documentos_por_mes`, la carpeta donde vive, `Location`, el formato, `Provider iceberg`, el dueño y las propiedades. `current-snapshot-id` es la página vigente, `format=iceberg/parquet` dice que los datos están en Parquet y `format-version=1` es la versión del formato de Iceberg.

Celda 4.2 · Los cajones que se armaron

La pregunta. ¿Cuántos cajones hay y cuántos documentos tiene cada uno?

Por qué ahora. Después de ver la regla, se ven los cajones que produjo.

En el almacén. Es recorrer los cajones y contar las facturas de cada uno.



fecha_emision (azul). La fecha real de un documento, por ejemplo el 15 de junio de 2024.

months(...) (amarillo). La transformación de la partición. Cuenta cuántos meses pasaron desde enero de 1970 hasta el mes de esa fecha.

fecha_emision_month (gris). El número del cajón. Para junio de 2024 es 653, porque de enero de 1970 a enero de 2024 hay 54 años de doce meses, 648 meses, y junio es cinco meses después.

add_months (verde). La función que usa la celda para convertir ese número de vuelta en una fecha legible, el primer día del mes.

La sentencia, parte por parte.

- `add_months(DATE '1970-01-01', partition.fecha_emision_month) AS mes` toma el número del cajón y lo suma como meses a enero de 1970, para mostrar el mes como fecha.
`partition.fecha_emision_month` es el número guardado del cajón.
- `record_count AS documentos` es cuántas filas tiene el cajón.
- `file_count AS archivos` es cuántos archivos tiene.
- `FROM mi_espacio.documentos_por_mes.partitions` lee la vista de sistema `.partitions`, con una fila por cajón.
- `ORDER BY mes` ordena de enero a diciembre.

```
sql
%%sql
-- Celda 4.2
SELECT add_months(DATE '1970-01-01', partition.fecha_emision_month) AS mes,
       record_count AS documentos,
       file_count AS archivos
FROM mi_espacio.documentos_por_mes.partitions
ORDER BY mes
```

Lo que sale en pantalla.

```
text
mes      documentos  archivos
2024-01-01 2500         1
2024-02-01 2500         1
2024-03-01 2500         1
2024-04-01 2500         1
2024-05-01 2500         1
2024-06-01 2500         1
2024-07-01 2500         1
2024-08-01 2500         1
2024-09-01 2500         1
2024-10-01 2500         1
2024-11-01 2500         1
2024-12-01 2500         1
12 filas.
```

Cómo se lee. Doce cajones, uno por mes de 2024, cada uno con 2500 documentos en un archivo. La fecha de la columna `mes` es siempre el día 1 porque representa el mes completo.

Paso 5 · Cambiar el criterio

Celda 5.1 · De mes a día

La pregunta. ¿Cómo se cambia el criterio de partición cuando un mes queda demasiado grande?

Por qué ahora. El volumen crece y conviene bajar de mes a día. Con otras tecnologías eso significa crear una tabla nueva y reescribir toda la historia. Aquí es una sentencia.

En el almacén. Es decidir que desde hoy las facturas nuevas se guardan en un cajón por día, sin mover las que ya están en los cajones por mes.

La sentencia, parte por parte.

- `ALTER TABLE documentos_por_mes` modifica la tabla particionada.
- `REPLACE PARTITION FIELD months(fecha_emision)` indica qué criterio se reemplaza.
- `WITH days(fecha_emision)` es el criterio nuevo, un cajón por día. Rige de aquí en adelante.

```
sql
%%sql
-- Celda 5.1
ALTER TABLE documentos_por_mes
REPLACE PARTITION FIELD months(fecha_emision) WITH days(fecha_emision)
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. El cambio se hizo al instante. No se movió ningún documento, solo cambió la regla.

Celda 5.2 · La regla nueva

La pregunta. ¿Qué dice ahora la tabla sobre su partición?

Por qué ahora. Para confirmar que el cambio quedó registrado.

En el almacén. Es volver a leer en la portada cómo se rotulan los cajones.

La sentencia, parte por parte.

- `DESCRIBE TABLE EXTENDED documentos_por_mes` pide otra vez la descripción completa de la tabla.

```
sql
%%sql
-- Celda 5.2
DESCRIBE TABLE EXTENDED documentos_por_mes
```

Lo que sale en pantalla.

```
text
col_name      data_type      comment
rut_emisor    string
razon_social_emisor string
tipo_dte      int
folio         bigint
fecha_emision date
fecha_recepcion timestamp
monto_neto    decimal(18,2)
monto_iva     decimal(18,2)
monto_total   decimal(18,2)
estado_sii    string

# Partitioning
Part 0        days(fecha_emision)

# Metadata Columns
_spec_id      int
_partition    struct<fecha_emision_month:int, fecha_emision_day:date>
_file         string
_pos          bigint
_deleted      boolean

# Detailed Table Information
Name          spark_catalog.mi_espacio.documentos_por_mes
Location      hdfs://namenode:8020/warehouse/iceberg/mi_espacio.db/documentos_por_mes
Provider      iceberg
Owner         root
Table Properties [current-snapshot-id=8267141702760514426,format=iceberg/parquet,format-version=1]
27 filas.
```

Cómo se lee. Part 0 ahora dice `days(fecha_emision)`. Y en `_partition` aparecen los dos criterios, `fecha_emision_month` y `fecha_emision_day`, porque la tabla recuerda el viejo para los archivos que ya estaban. El resto de la descripción es igual a la de la celda 4.1.

Celda 5.3 · Cargar un mes nuevo

La pregunta. ¿Con qué criterio se guardan los documentos que llegan después del cambio?

Por qué ahora. Para ver el efecto del cambio hace falta escribir algo nuevo.

En el almacén. Es guardar las facturas de enero de 2025 en la bodega que ahora se ordena por día.

La sentencia, parte por parte.

- `INSERT INTO documentos_por_mes` agrega filas a la tabla particionada.
- `SELECT * FROM curso.dte_2025_enero` son los documentos de enero de 2025, en el espacio `curso`.
- `ORDER BY fecha_emision` los hace llegar agrupados por fecha, como pide una tabla particionada.

```
sql
%%sql
-- Celda 5.3
INSERT INTO documentos_por_mes
SELECT * FROM curso.dte_2025_enero ORDER BY fecha_emision
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. El mes nuevo entró.

Celda 5.4 · Los archivos de cada criterio

La pregunta. ¿Se reescribieron los documentos que ya estaban?

Por qué ahora. Es la prueba de que cambiar el criterio no reescribe la historia.

En el almacén. Es contar cuántos cajones hay de cada tipo de rótulo.

La sentencia, parte por parte.

- `SELECT spec_id` es el número del criterio con que se escribió cada archivo. El 0 es el criterio por mes y el 1 el criterio por día.
- `count(*) AS archivos` cuenta los archivos de cada criterio.
- `sum(record_count) AS documentos` suma sus filas.
- `FROM mi_espacio.documentos_por_mes.files` es la vista de archivos.
- `GROUP BY spec_id` junta los archivos por criterio.
- `ORDER BY spec_id` ordena del criterio viejo al nuevo.

```
sql
%%sql
-- Celda 5.4
SELECT spec_id,
       count(*) AS archivos,
       sum(record_count) AS documentos
FROM mi_espacio.documentos_por_mes.files
GROUP BY spec_id
ORDER BY spec_id
```

Lo que sale en pantalla.

```
text
spec_id  archivos  documentos
0         12      30000
1         31      2500
2 filas.
```

Cómo se lee. Con el criterio 0, por mes, siguen los 12 archivos con los 30000 documentos de 2024, intactos. Con el criterio 1, por día, están los 2500 documentos de enero de 2025, repartidos en 31 archivos, uno por día. Los dos criterios conviven en la misma tabla.

Celda 5.5 · Un mes del período viejo

La pregunta. ¿Sigue funcionando la consulta de junio de 2024?

Por qué ahora. Hay que comprobar que convivir con dos criterios no rompió las consultas sobre lo viejo.

En el almacén. Es pedir el total de junio a la bodega que ahora tiene cajones de dos tipos.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos, sum(monto_total) AS total_junio` cuenta y suma.
- `FROM documentos_por_mes` es la tabla particionada.
- `WHERE fecha_emision BETWEEN DATE '2024-06-01' AND DATE '2024-06-30'` es junio de 2024.

```

sql
%%sql
-- Celda 5.5
SELECT count(*)          AS documentos,
       sum(monto_total) AS total_junio
FROM documentos_por_mes
WHERE fecha_emision BETWEEN DATE '2024-06-01' AND DATE '2024-06-30'

```

Lo que sale en pantalla.

```

text
documentos  total_junio
2500        24547341762.59
1 fila.

```

Cómo se lee. El mismo resultado de siempre, 2500 documentos y 24547341762.59.

Celda 5.6 · Un mes del período nuevo

La pregunta. ¿Y un mes guardado con el criterio nuevo?

Por qué ahora. Para ver que la misma forma de preguntar sirve para los dos criterios.

En el almacén. Es pedir el total de enero de 2025, que está en cajones por día.

La sentencia, parte por parte.

- `SELECT count(*) AS documentos, sum(monto_total) AS total_enero_2025` cuenta y suma.
- `FROM documentos_por_mes` es la misma tabla.
- `WHERE fecha_emision BETWEEN DATE '2025-01-01' AND DATE '2025-01-31'` es enero de 2025.

```

sql
%%sql
-- Celda 5.6
SELECT count(*)          AS documentos,
       sum(monto_total) AS total_enero_2025
FROM documentos_por_mes
WHERE fecha_emision BETWEEN DATE '2025-01-01' AND DATE '2025-01-31'

```

Lo que sale en pantalla.

```

text
documentos  total_enero_2025
2500        24895193513.17
1 fila.

```

Cómo se lee. 2500 documentos que suman 24895193513.17. La consulta no tuvo que saber que ese mes está guardado por día. El motor sabe qué criterio usó cada archivo y usa el que corresponde.

Preguntas frecuentes

¿Por qué la consulta del paso 1 no fue lenta si leyó todo?

Porque la tabla es chica, 0,78 MB. Aquí lo que se mide no es el tiempo sino cuánto se lee. Con años de historia y millones de documentos, leer doce veces más es la diferencia entre segundos y minutos.

¿Qué es el rótulo de un papelito?

El mínimo y el máximo de cada columna dentro del archivo. Iceberg los guarda en sus manifiestos, fuera del archivo, y por eso puede descartar un archivo sin abrirlo. En la vista `.files` se ven en `readable_metrics`, como `lower_bound` y `upper_bound`.

¿Tengo que filtrar por el mes para que el motor use los cajones?

No. Ese es el particionamiento oculto. Filtras por `fecha_emision`, la columna real, e Iceberg sabe cómo se deriva el cajón a partir de ella. Si particionas por una columna aparte, como un `anio_mes` calculado, cualquier consulta que no la nombre lee la tabla completa.

¿Qué es el número 653 y por qué la celda no lo muestra?

Es cómo guarda Iceberg un cajón mensual, la cantidad de meses desde enero de 1970. Enero de 2024 es 648, porque son 54 años de doce meses, y cada mes siguiente suma uno. La celda 4.2 lo convierte en fecha con `add_months` para que se lea fácil.

¿Por qué la celda 2.1 necesita el ORDER BY?

Porque al escribir en una tabla particionada las filas tienen que llegar agrupadas por cajón. Sin el orden, las filas de un mismo mes llegan mezcladas con las de otros y la escritura falla.

¿Particionar siempre conviene?

No siempre. Particionar crea más archivos, doce aquí y treinta y uno para un solo mes al bajar a día. Muchos archivos chicos tienen su propio costo. Se particiona por una columna con pocos valores distintos que aparezca en casi todas las consultas, como la fecha.

¿Qué pasa con los documentos viejos si nunca se reescriben?

Se quedan con el criterio por mes y se siguen consultando bien, como mostró la celda 5.5. Si algún día conviene que también queden por día, se pueden reescribir aparte, pero no es obligatorio.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Tres ideas distintas. Particionar guarda los datos agrupados para no leerlos todos. El particionamiento oculto te deja filtrar por la columna real. La evolución de particiones cambia el criterio sin reescribir lo que ya estaba.

En el almacén. La bodega ahora tiene pasillos rotulados. Quien pide junio no necesita saber cómo está ordenada, y cuando un pasillo quedó chico se dividió sin mover nada de lo guardado.

Particionar
PARTITIONED BY
months(fecha_emision)

Particionamiento oculto
se filtra por fecha_emision
nunca por el mes

Evolución de particiones
REPLACE PARTITION FIELD
lo viejo no se reescribe

Particionar (azul). Un cajón por mes, para que una consulta de un mes lea un solo cajón.

Particionamiento oculto (morado). La consulta nombra la fecha real y el motor sabe a qué cajón ir.

Evolución de particiones (verde). El criterio cambió de mes a día y los archivos viejos siguieron con el suyo.

Los números de la solución ejecutada.

Tabla	Archivos a leer para junio	Filas a leer	Bytes a leer
Sin particionar	1	30.000	776.514
Particionada por mes	1	2.500	71.779

30.000 filas leídas contra 2.500 para responder exactamente la misma pregunta, y sin que la consulta cambiara una letra.